seelo 서비스 명세서

- 문서 역할: 기획서의 기능을 어떤 기술로 어떻게 구현하고 검증할지 정의하는 기술 기준 문서
- 기능 기준: seelo 기획서 — 범위(8장), 화면(9장), 기능(10장, F-01~F-12), 작업 분해(11장, W-01~W-12), 지도별 정비 방법(15장)
- 대상 버전: 베타
- 2026-10-23 개정: 구현 스택을 React(화면)·FastAPI(API)·PostgreSQL·Procrastinate(작업 큐)로 바꿨다. 1·2장, 기능별 경로·작업 큐 문구, 5·6장, 13장 테스트 문구, 12장 결정 사항을 이에 맞췄다.
- 2026-10-20 개정: 서비스명을 seelo로 변경했다. 기존 파일 경로와 MVP 디렉터리명은 연결 호환을 위해 유지한다.
- 우선순위 표기: 기획서 8장과 같다(`P0` 베타에 필수, `P1` 일정이 허용되면, `P2` 후속 개발).
- 이 문서는 기능 규칙을 다시 적지 않는다. 기능 규칙이 바뀌면 기획서를 먼저 고치고, 이 문서의 구현 방법을 맞춘다.
- 2026-10-17 개정: 기획서의 자동 정비 구조(무료 진단 → 자동 정비 신청 → Google·카카오 연결 → 정비안 승인 → 자동 정비 → 2주 뒤 재측정 → 전후 비교 보고서)에 맞춰 고쳤다.

1. 시스템 구성

1.1 기술 스택

영역	기술	용도
화면	React + TypeScript + Vite, React Router, TanStack Query, react-hook-form + zod, Tailwind CSS + shadcn/ui, Recharts	S01~S16·A01 화면(SPA), 입력 검증, 진행률 조회, 지표 차트
API	FastAPI + Pydantic v2, httpx	JSON API(`/api/...`), 요청·응답 스키마 검증, LLM 구조화 출력 검증, 외부 API 호출
인증	서버 세션(`sessions` 테이블)과 세션 쿠키, Authlib(구글·카카오 OAuth), Argon2id 비밀번호 해시(argon2-cffi), CSRF 헤더 검사	가입·로그인·세션·접근 제어
데이터	PostgreSQL(16 이상) + SQLAlchemy 2.0 + Alembic + psycopg 3	관계형 데이터, 이력, 마이그레이션. 원본 응답은 JSONB
비동기 작업	Procrastinate(PostgreSQL 기반 작업 큐)	정보 수집, 반복 AI 호출, 정비안 생성, 자동 정비, 재측정, 보고서 생성, 이메일 발송, 주기 작업
AI	OpenAI API(웹 검색 도구), Gemini API(Google 검색 그라운딩)	측정, 추천 추출, 질문 생성, 정보 수집용 웹 검색, 정비안 문장 생성
외부 데이터	Google Places API(New), 카카오 로컬 API, NAVER API HUB 지역 검색, 인스타그램 Graph API(PoC 후)	가게 식별, 기본정보·상세정보 수집
지도 정비	브라우저 자동화(도구는 D-13), Google Business Profile API(승인 후)	Google·카카오 연결 확인, 현재 값 읽기, 정비, 반영 확인
이메일	트랜잭션 이메일 서비스(SMTP, D-08). 개발 환경은 Mailpit	인증, 비밀번호 재설정, 진단·정비·보고서 알림
배포	Docker Compose(`web`·`api`·`worker`·PostgreSQL)	개발·운영 환경 일치. Redis는 쓰지 않는다
관측	구조화 로그, 오류 추적, 작업별 비용 기록	실패 원인, 응답 시간, API 비용 확인

- 화면과 API는 같은 도메인에서 서비스한다. 세션 쿠키를 `SameSite=Lax`로 두기 위해서다. 개발 환경에서는 Vite 개발 서버가 `/api` 요청을 FastAPI로 넘긴다.
- 경로 표기: 3장의 경로 중 `/api/`로 시작하는 것은 FastAPI 경로이고, 나머지는 React 화면 경로다.

- 필드 이름: API JSON 필드는 협업 규칙(기획서 13.4)대로 `camelCase`로 쓴다(Pydantic 별칭). PostgreSQL은 따옴표 없는 이름을 소문자로 바꾸므로 DB 컬럼은 `snake_case`로 만든다. 6장의 컬럼 이름은 API 필드 이름으로 적는다.
- 화면용 TypeScript 타입은 FastAPI가 만든 OpenAPI 문서에서 생성한다(openapi-typescript). API 응답 필드를 바꾸면 타입을 다시 생성해 화면과 맞춘다.
- LLM 구조화 출력(F-04 AI 웹 검색, F-05, F-07, F-09)은 Pydantic 모델로 스키마를 정하고 검증한다.
- 랜딩(S01)의 3D 스크롤 장면은 `05_발표자료/결과물/한눈_랜딩시안_프로페셔널_20261016`의 three.js 코드를 React 컴포넌트로 감싸 쓴다.
- P0에서는 파일 저장소를 쓰지 않는다. 사진은 저장하지 않고(기획서 F-04), P1 전후 비교 보고서 PDF를 만들 때 필요하면 오브젝트 스토리지를 추가한다.

1.2 처리 흐름

```
브라우저(React 화면)
  → web(nginx): React 빌드 파일 제공, /api 요청 전달
      → api(FastAPI)
          → PostgreSQL: 서비스 데이터, 작업 큐(작업 등록)
              → worker(Procrastinate, 작업을 꺼내 실행)
                  → Google Places API / 카카오 로컬 API / NAVER API HUB
                  → 가게 공식 웹사이트(robots.txt 허용 시)
                  → OpenAI API / Gemini API
                  → 인스타그램 Graph API(PoC 통과 시)
                  → Google 비즈니스 프로필 관리 화면·카카오 파트너센터(브라우저 자동화)
                  → Google Business Profile API(승인 후)
                  → 이메일 서비스
```

1.3 프로세스와 큐

프로세스·큐	하는 일
`web`	nginx. React 빌드 파일을 주고, `/api/`로 시작하는 요청을 `api`로 넘긴다. 그 밖의 경로는 `index.html`을 돌려줘 React 라우터가 화면을 고르게 한다.
`api`	FastAPI. 요청·권한 검사, 작업 등록, 진행률 조회, 화면에 필요한 JSON 응답. 가게 식별(F-03)을 빼고는 외부 API를 직접 호출하지 않는다.
worker `collect`	가게 식별 이후 정보 수집(진단·재측정)
worker `measure`	질문 생성, 추천 추출, 지표 계산, 고쳐야 할 점 판정(진단·재측정)
worker `ai_openai`·`ai_gemini`	AI 반복 호출(`callAi`). 공급자별 큐로 나누고, 큐마다 워커 동시 실행 수로 F-06의 공급자별 동시 호출 제한(기본 5)을 맞춘다.
worker `plan`	정비안 생성·사실 검사
worker `fix`	연결 확인, 현재 값 읽기, 자동 정비, 반영 확인. seelo 공용 계정 하나로 동시에 여러 브라우저를 열지 않도록, 같은 지도의 작업은 Procrastinate `lock`(`fix:google`·`fix:kakao`)으로 한 번에 하나만 실행한다.
worker `report`	전후 비교 보고서 생성
worker `email`	이메일 발송
주기 작업	별도 scheduler 프로세스 없이 Procrastinate 주기 작업(`@app.periodic`, cron)으로 등록한다. 워커가 여러 개여도 주기마다 한 번만 등록되고, 워커가 하나도 없으면 등록되지 않는다. 미인증 계정 정리, 만료 스냅샷 삭제, 초대

프로세스·큐	하는 일
	미도착 확인, 반영 대기 칸 다시 확인, 재측정 예약 실행, 멈춘 작업 다시 시도

- 큐는 위처럼 나누되, 베타에서는 워커 프로세스 하나가 여러 큐를 맡아도 된다. `fix` 큐 워커는 관리자 로그인 브라우저 프로필(D-13)이 있는 디스크가 필요해 따로 띄운다.
- 가게 식별(F-03)은 S08에서 결과를 바로 보여 줘야 하므로 API 요청 안에서 처리한다. 지도 API는 httpx 비동기로 동시에 부르고, 외부 호출마다 5초 제한을 두며, 전체 10초를 넘기면 "잠시 후 다시 시도" 오류를 낸다.

2. 권한과 보안 모델

역할	접근 범위
비회원	S01, S02~S06
사장님(`owner`)	본인 계정·가게·측정·정비 신청·정비안·정비 기록·전후 비교 보고서(S07~S16)
관리자(`admin`)	A01(`/admin/...`): 신청 승인·거절, 연결 확인, 수동 처리 완료, 정비안·보고서 다시 생성, 지금 재측정

- 가게·측정·정비안·정비 기록·보고서를 읽거나 바꾸는 모든 API 요청은 서버에서 소유자(`stores.ownerId = 현재 사용자`) 또는 관리자 여부를 확인한다. URL의 ID만 바꿔서 남의 데이터를 볼 수 없어야 한다.
- 접근 제어는 API가 한다. React 화면에서 숨기거나 다른 화면으로 보내는 것은 접근 제어로 보지 않는다.
- 무료 진단 API(S11) 응답에는 고쳐야 할 점의 개수만 넣는다. 판정 내용(`findings`)은 정비안 API(S13)에서만 응답한다.
- 정비안(S13)은 그 가게의 정비 신청이 승인되고 해당 지도가 연결된 뒤에만 응답한다. 전후 비교 보고서(S16)는 보고서가 완료된 뒤에만 응답한다. 조건이 맞지 않으면 API 응답에 내용을 넣지 않는다.
- API 응답은 화면별 Pydantic 응답 모델로 만든다. 응답 모델에 없는 필드는 내보내지 않으므로, 응답하면 안 되는 필드(예: S11의 `findings` 내용)는 그 화면의 응답 모델에 두지 않는다.
- 자동 정비 작업에는 사장님이 승인한 칸의 승인 값(`fixPlanItems.approvedValue`)만 넘긴다. 승인되지 않은 칸을 고치는 코드 경로를 두지 않는다.
- 사장님 아이디·비밀번호는 받지 않는다. seelo 공용 Google·카카오 계정의 접속 정보와 Google API 토큰은 환경변수(비밀 값)로만 넣고 코드·DB·로그에 남기지 않는다.

2.1 세션·CSRF·화면 접근

항목	방법
세션	로그인하면 32바이트 난수 세션 토큰을 만들어 `sessions`에 해시로만 저장하고, 쿠키(`Secure`·`HttpOnly`·`SameSite=Lax`, 14일)로 준다. 로그아웃하면 그 세션 행을 지운다. 비밀번호를 재설정하면 그 사용자의 모든 세션 행을, 변경(S14)하면 지금 세션을 뺀 나머지 세션 행을 지운다. 탈퇴하면 계정과 함께 지워진다
CSRF	세션을 만들 때 CSRF 토큰도 만들어 `sessions`에 해시로 저장하고, 화면 코드가 읽을 수 있는 쿠키(`csrf_token`, `HttpOnly` 아님)로 준다. 상태를 바꾸는 요청(POST·PUT·PATCH·DELETE)은 `X-CSRF-Token` 헤더가 세션의 토큰과 같아야 하고, 다르거나 없으면 403. 로그인 전 요청(가입·로그인·비밀번호 재설정)은 `GET /api/auth/csrf`로 받은 임시 토큰을 같은 방식으로 쓴다
OAuth	`state`는 Authlib으로 만들고 콜백에서 검증한다. 콜백은 GET 리디렉션이라 CSRF 헤더 대신 `state`로 막는다. 구글·카카오에 등록하는 리디렉션 주소는 `https://<seelo 도메인>/api/oauth/<provider>/callback`

항목	방법
응답 코드	로그인이 필요하면 401, 권한이 없으면 403, 없는 대상이나 아직 응답할 수 없는 대상은 404
화면 이동	React 라우터가 `GET /api/auth/me`로 로그인 여부를 확인한다. 로그인하지 않았으면 S05로 보내고 로그인 후 원래 화면으로 돌려보낸다. 로그인한 사장님이 S01·S05에 오면 S07로 보낸다(기획서 9.2). API가 401을 주면 S05로 보낸다
관리자	관리자 API는 `/api/admin/...`에 모으고 `users.role = admin`이 아니면 403. A01 화면 경로는 `/admin/...`
CORS	화면과 API를 같은 출처로 서비스하므로 CORS를 열지 않는다

3. 기능별 구현

F-01 계정

구현 요소	내용
화면 경로	`/signup`, `/login`, `/verify-email/<token>`, `/signup/kakao-email`, `/password/forgot`, `/password/reset/<token>`, `/account`
API	`POST /api/auth/signup`, `POST /api/auth/login`, `POST /api/auth/logout`, `GET /api/auth/me`, `GET /api/auth/csrf`, `POST /api/auth/verify-email`, `POST /api/auth/verify-email/resend`, `POST /api/auth/kakao-email`, `POST /api/auth/password/forgot`, `POST /api/auth/password/reset`, `POST /api/account/password`, `DELETE /api/account`, `GET /api/oauth/<provider>/start`, `GET /api/oauth/<provider>/callback`
인증·재설정 링크	메일 링크는 화면 경로(`/verify-email/<token>`, `/password/reset/<token>`)로 연다. 화면이 토큰을 API에 POST해 처리한다
OAuth 콜백	세션을 만든 뒤 화면 경로로 보낸다: 처음 가입이면 약관 동의, 카카오가 이메일을 주지 않았으면 S04, 그 밖에는 S07
이메일 정규화	앞뒤 공백 제거, 소문자 변환 후 저장·비교
중복 검사	인증을 마친 계정(`emailVerifiedAt` 있음)과 이메일이 같으면 가입 차단하고 `authIdentities.provider`로 기존 가입 방식을 안내. 같은 이메일의 미인증 계정만 있으면 그 계정을 지우고 새로 만든다
비밀번호	Argon2id 해시만 저장. 소셜 가입 계정은 `passwordHash` 없음
인증·재설정 토큰	32바이트 난수. DB에는 해시만 저장, `expiresAt`(인증 24시간, 재설정 1시간), `usedAt`으로 1회용 처리
구글	OpenID Connect. `email_verified=true`인 이메일만 사용
카카오	이메일 동의 항목 요청. 이메일을 받지 못하면 임시 상태로 S04에서 이메일 입력·인증 후 가입 완료
로그인 잠금	`failedLoginCount` 5회 연속이면 `lockedUntil = 지금 + 10분`, 성공 시 0으로 초기화

구현 요소	내용
세션	서버 세션(`sessions`, 2.1). 쿠키 `Secure`, `HttpOnly`, `SameSite=Lax`, 유지 14일
탈퇴	진행 중인 측정·정비 작업과 재측정 예약을 먼저 취소한 뒤, 한 트랜잭션에서 계정·가게·측정·정비 신청·정비안·정비 기록·보고서 삭제. 이미 고친 Google·카카오 값은 지도에 남는다. 무료 진단을 쓴 계정이면 `freeUsageLedger`에 `HMAC-SHA256(서버 비밀키, 이메일)`만 남김
무료 사용 확인	가입·측정 시작 때 이메일 HMAC 값이 `freeUsageLedger`에 있으면 무료 사용 완료로 본다

F-02 가게 등록·변경

- 계정당 보관 중이 아닌 가게는 1곳(`stores`에 `archivedAt IS NULL` 조건의 `ownerId` 부분 고유 인덱스).
- 변경: 기존 가게에 `archivedAt`을 기록하고, 진행 중인 측정 작업은 Procrastinate 작업을 취소(5장)한 뒤 `measurementRuns.status = canceled`.
- 정비 신청이 진행 중(`fixRequests.status`가 `rejected`·`report_ready`가 아님, 7장)이면 변경 요청을 409로 거부한다.
- 무료 진단을 이미 쓴 계정은 새 가게의 `collectStore`까지만 실행하고 가게를 `collected`로 둔다. 측정은 자동 정비 신청이 승인된 뒤 시작한다.
- 업종 판정: Google `primaryType`·`types`, 카카오 `category_group_code`·`category_name`, 네이버 `category`를 음식점·카페·바 목록과 비교한다. 목록(예: Google `restaurant` 계열·`cafe`·`coffee_shop`·`bar`, 카카오 `FD6`·`CE7`)은 사전 확인(PoC)에서 확정한다.
- 무료 차감: `measurementRuns.isFree = true`인 측정이 `completed`가 되는 순간 `freeUsageLedger`에 기록한다. `failed`·`canceled`는 차감하지 않는다.

F-03 링크 판별과 가게 식별

- 입력 문자열에서 URL을 정규식으로 뽑는다. 공유 문구를 통째로 붙여넣어도 첫 번째 지원 URL만 쓴다.
- 짧은 링크(`maps.app.goo.gl`, `kko.to`, `naver.me`)는 리다이렉트를 자동으로 따라가지 않고 `Location` 헤더만 읽는다. 최대 3번, 한 번에 5초.
- 지도별 식별

지도	링크에서 얻는 값	확인 방법
Google	장소 이름, 좌표(`@위도,경도`)	Places Text Search(가게 이름, 좌표 주변 200m)로 장소 ID 확정
카카오	장소 번호	키워드 검색(사장님이 입력한 가게 이름) 결과 중 `id`가 장소 번호와 같은 것. 없으면 이름이 비슷한 결과를 S09 후보로, 그것도 없으면 다른 링크 요청
네이버	장소 번호(API 응답에 장소 번호가 없어 직접 대조 불가)	지역 검색(가게 이름) 결과 중 정규화한 이름이 같은 결과가 1개면 확정, 여러 개면 S09, 없으면 다른 링크 요청

- 다른 지도에서 같은 가게 찾기: 가게 이름으로 검색해 좌표 거리 100m 이내이고 이름 유사도가 기준 이상인 결과를 같은 가게로 본다. 유사도 계산은 F-07 이름 정규화 후 비교하며, 기준값은 사전 확인(PoC)의 정확도 측정으로 정한다.
- 결과는 `storeSources`에 지도별 `externalId`, `url`, `matchStatus(found·not_found·candidate)`로 저장한다. Google·카카오가 `not_found`면 F-08 "지도에 정보 없음"으로 판정하고, 그 지도의 정비안은 만들지 않는다(F-09).

F-04 정보 수집

- 작업: `collectStore(storeId, runId)`, 전체 30초 목표. 출처마다 따로 실행하고 하나가 실패해도 나머지 결과는 저장한다. 재측정(F-11) 때도 같은 작업을 다시 실행한다.
- 출처별 처리

출처	호출	저장
Google Places	Place Details(필요한 필드만 field mask로 요청)	장소 ID는 계속 저장. 나머지는 `sourceSnapshots`(보관 기한은 기획서 15장 미정 3)
카카오 로컬	키워드 검색	장소 ID·장소 링크만 저장. 화면 표시·비교 때마다 다시 호출
네이버 지역 검색	NAVER API HUB 지역 검색	결과는 가게 식별, 홈페이지 주소 확인, 기본정보(이름·주소·전화·업종) 비교에만 쓴다. 비교는 코드로 하고 응답 내용은 AI 입력에 넣지 않음(약관 확인 전)
공식 웹사이트	`robots.txt` 확인(사용자 에이전트 `SeeloBot`) 후 첫 페이지와 메뉴로 보이는 링크 1개, 페이지당 1MB·10초 제한	본문 텍스트에서 메뉴·가격·영업시간·소개·영문 페이지 여부를 LLM으로 추출해 저장
AI 웹 검색	OpenAI·Gemini 각 1회, 웹 검색을 켜고 "가게 이름·주소에 대한 정보"를 구조화 출력(JSON)으로 요청	`webEvidence`에 사이트 주소·이름·사이트별 가게 정보 저장
인스타그램	기본: AI 웹 검색 결과 중 `instagram.com` 주소 사용. 추가: Business Discovery(기능 플래그, PoC 통과 시 켬)	계정 주소, 소개글, 최근 게시물 글·게시일

- 카카오의 영업시간·메뉴 등 상세 값은 진단 수집에서 읽지 않는다. 연결 뒤 `fix` 워커가 파트너센터 매장정보 화면에서 읽는다(F-10). 이렇게 읽은 값은 정비안의 칸별 현재 값과 정비 기록의 고치기 전 값으로만 저장하고, 그 밖의 저장 범위는 D-10에서 정한다.
- 로그인 세션으로 사이트를 여는 수집 도구, 지도 페이지 크롤링, 차단 우회는 쓰지 않는다. 정비용 관리 화면 자동화는 F-10의 범위로 한정한다.
- (P1) 메뉴판 OCR: Google 사진을 불러와 비전 모델로 메뉴판인지 판별하고 글자만 추출한다. 사진 파일은 저장하지 않는다. Google 약관 확인(기획서 13장 PoC) 전에는 켜지 않는다.

F-05 질문 생성

- 저가 LLM 1회 호출, JSON 구조화 출력(`lang`, `intentType`, `text` 3개).
- 코드 검증: 가게 이름(한글·영문, 정규화 후 부분 일치) 미포함, 지역 이름·업종 포함, 3개 문장 서로 다름. 실패하면 3번까지 다시 생성하고, 그래도 실패하면 업종별 고정 템플릿 사용.
- `questionSets.version`과 함께 저장하고, 측정 작업은 질문 세트 ID를 참조한다.
- 재측정은 질문을 새로 만들지 않고 정비 전 측정의 `questionSetId`를 그대로 쓴다.

F-06 AI 측정

- 작업: `runMeasurement(runId)`가 60개의 `callAi` 작업을 만든다(질문 3 × AI 2 × 10회).
- `measurementRuns.kind`로 진단(`diagnosis`)과 재측정(`remeasure`)을 구분한다. 재측정은 무료 차감 대상이 아니다(`isFree = false`).
- 동시 실행: 공급자별 동시 호출 수 제한(기본 5, 환경변수로 조정).
- 호출: 질문 문장만 보내고 시스템 지시는 붙이지 않는다. OpenAI는 웹 검색 도구, Gemini는 Google 검색 그라운딩을 켠다.
- 제한 시간 60초, 실패 시 2번까지 지수 백오프로 다시 시도. `aiExecutions`의 (`runId`, `questionId`, `provider`, `repeatNo`) 고유 키로 중복 호출과 중복 비용을 막는다.
- 저장: `aiExecutions`(모델 ID, 회차, 상태, 지연 시간, 오류 코드, 시도 횟수), `aiResponses`(답변 원문, 원본 JSON, 출처 URL).
- 모든 호출이 끝나면 실패율을 계산해 30%를 넘으면 `failed`, 아니면 다음 단계(F-07)로 넘어간다.
- 다시 시도는 가게당 하루 3회까지다(진단·재측정 공통). 그날(한국 시간 기준) 해당 가게의 다시 시도로 만든 `measurementRuns` 수로 판단하고, 넘으면 429로 거부한다.
- 수집에서 지역·업종을 얻지 못하면 측정을 만들지 않고 가게를 `measure_failed`로 둔다(무료 차감 없음).

- 진행률: `measurementRuns.progressDone/progressTotal`을 갱신하고, S10은 `GET /api/runs/<id>/progress`를 3초마다 조회한다(TanStack Query `refetchInterval`, 완료·실패면 조회를 멈춤). 재측정도 같은 API를 쓴다.

F-07 추천 추출과 지표

- 답변마다 저가 LLM으로 추천 목록을 뽑는다. 출력 JSON: `recommendations[{name, rank, sourceUrls}]`. 추출에 실패한 답변은 원문을 남기고 지표 계산에서 뺀 뒤 건수를 기록한다.
- 이름 정규화: 소문자 변환, 공백·기호 제거, 지점 표기("본점", "○○점", "branch" 등) 제거. 대상 가게 이름 후보는 세 지도의 한글·영문 이름이다.
- 대상 가게 판정: 정규화한 이름이 후보 중 하나와 같으면 등장(`recommendations.isTarget = true`).
- 지표는 `metrics`에 공급자·질문·대상(우리 가게·주변 가게)별로 값과 분모를 함께 저장한다. 계산식은 기획서 F-07을 따른다.
- 주변 가게 상위 5곳은 정규화 이름으로 묶어 집계한다.

F-08 고쳐야 할 점 판정

- 네이버·카카오 기본정보 비교는 정규화(공백·기호·지점명 제거, 전화번호 숫자만) 후 코드로 한다. 카카오 로컬 API 값은 저장하지 않고 판정 때 다시 불러온다.
- 코드 규칙 엔진으로 판정한다(LLM 사용은 "질문 조건 정보 부족", "AI가 틀리게 말함"의 문장 비교에 한정).
- Google 지도·카카오맵에서 고칠 수 있는 항목만 만든다. 인스타그램·외부 근거처럼 두 지도 밖에서 고쳐야 하는 항목은 판정하지 않는다.
- 결과는 `findings`에 단계(`diagnosis·plan`), 분류, 제목 키, 대상 지도·칸, 영향 순서(주변 가게 중 그 정보를 갖춘 수, 관련 질문 수), 근거(관련 응답 ID·수집값·주변 가게)를 저장한다.
- 무료 진단에서는 카카오 기본정보만 쓴다(`stage = diagnosis`). 연결 뒤 읽은 카카오·Google 현재 값으로 정비안을 만들 때 판정을 추가한다(`stage = plan`, F-09).
- S11에는 `stage = diagnosis`인 `findings`의 개수만 응답한다.

F-09 정비안 생성

- 화면 경로: S13 `/fix/plans/<planId>`. API: 조회 `GET /api/fix/plans/<planId>`, 승인 `POST /api/fix/plans/<planId>/approve`
- 작업: `generateFixPlan(fixRequestId, provider)`. 지도가 연결되고(`mapConnections.status = connected`) 현재 값 읽기(`readCurrentValues`)가 끝나면 지도별로 실행한다. 진단 결과가 없는 가게(무료 사용 후 가게 변경)는 신청 승인 직후 `collectStore→runMeasurement`부터 실행하고, 측정이 끝난 뒤 만든다.
- 입력: `sourceSnapshots·webEvidence`, 연결 뒤 읽은 현재 값, 기준 측정(`baseRunId`)의 `metrics·findings`
- 처리 순서
  1. 연결 뒤 읽은 현재 값으로 F-08 판정을 추가한다(`findings.stage = plan`).
  2. 칸 사전(지도별 바꿀 수 있는 칸, 칸 이름·글자 수·형식, D-17)에 있는 칸만 대상으로 한다. 가게 이름·주소·업종은 칸 사전에 넣지 않아 만들 수 없게 한다.
  3. LLM은 칸별 바꿀 값 문장(영문 소개, 한 줄 소개, 메뉴 설명 등)만 만든다. JSON 구조화 출력: `items[{field, proposedValue, findingIds, evidence}]`
  4. 코드 사실 검사: 영업시간·가격·전화 같은 사실 값이 입력 사실 목록에 없으면 `[ ]`로 바꾸고 `needsOwnerInput = true`("확인 필요"). 알레르기·할랄 정보는 입력 사실에 있을 때만 남기고 `sensitive = true`("사장님 확인 필요").
  5. 글자 수·형식을 넘는 값은 줄여서 1번 다시 만든다. 칸 사전에 없는 칸이 나오면 버린다.
- 생성 목표 3분. 실패하면 1번 자동 재시도, 그래도 실패하면 `fixPlans.status = generation_failed`로 두고 A01 [다시 생성]으로 다시 실행한다.
- 정비안 준비가 끝나면 `fixPlans.status = awaiting_approval`, 정비안 준비 이메일을 보낸다.
- 승인(S13 [정비 시작])
  - 한 트랜잭션에서 칸별 `approvedValue`(사장님이 고친 값 포함)와 제외 여부를 저장한다. 사장님이 고친 값도 칸 사전의 글자 수·형식 검사를 거친다.
  - `needsOwnerInput` 칸이 비어 있고 제외되지도 않았거나, `sensitive` 칸에 확인 체크가 없으면 400으로 거부한다.
  - `fixPlans.status`를 `awaiting_approval`에서 `approved`로 바꾸는 조건부 UPDATE로 중복 승인을 막는다(영향 행이 0이면 이미 승인된 것). 성공한 요청만 `applyFix`를 등록한다.
- Google·카카오에서 가게를 찾지 못했으면 그 지도의 정비안은 만들지 않고 S13 탭에 등록·인증 방법을 보여 준다. 등록·연결 뒤 `generateFixPlan`을 다시 실행한다.
- 한쪽 지도만 연결된 상태로 승인하면 그 지도만 정비한다. 나중에 다른 지도가 연결되면 그 지도의 `fixPlans`만 새로 만들어 승인받는다(정비 완료 시점은 D-16).
- 정비안은 생성일, 기준 진단(`baseRunId`), 버전, 사장님이 승인한 최종 값을 함께 저장한다.

F-10 연결과 자동 정비

- 화면 경로: S15 `/fix/connect`. API: 조회 `GET /api/fix/connect`, [초대했어요] `POST /api/fix/connect/<provider>/invited`

연결

단계	처리
초대 표시	사장님이 [초대했어요]를 누르면 `mapConnections.status = invited`, `invitedAt` 기록
초대 수락	관리자가 seelo 계정으로 초대를 수락하고 A01 [연결 확인]을 누른다. Google API 승인 뒤에는 `fix` 워커가 Account Management API로 초대를 조회·수락한다
연결 확인	`verifyConnection(connectionId)`: Google은 프로필 편집 화면이 열리는지(API 승인 뒤에는 `getVoiceOfMerchantState`의 `hasVoiceOfMerchant`), 카카오는 파트너센터에서 매장이 보이고 매장 > 매장정보 화면이 열리는지 확인
결과	정상이면 `connected`, 연결 완료 이메일, `readCurrentValues` → `generateFixPlan`. 인증 필요·검토 중·중복 프로필·대표 권한 없음이면 `needs_attention`과 사유를 저장하고 S15에 안내
초대 미도착	주기 작업 `checkPendingInvites`(매시간): `invitedAt`에서 3일이 지나도 `connected`가 아니면 S15 안내를 다시 띄우고 이메일을 1번 보낸다(`inviteReminderSentAt`)
권한 회수	작업이 지도를 열 때(연결 확인·현재 값 읽기·정비·반영 확인·재측정) 가게가 보이지 않거나 API가 403·404를 주면 `revoked`. 그 지도의 남은 칸은 `disconnected`로 끝내고 S07에 "연결이 끊겼어요"

자동 정비

- 작업: `applyFix(fixPlanId)`. 승인된 정비안 하나(지도 하나)를 브라우저 세션 하나로 처리한다.
- 칸마다 순서: 현재 값을 다시 읽어 `fixLogs.beforeValue`에 먼저 저장 → 승인 값 입력·저장 → 다시 읽어 확인 → `applied` 또는 `pending_reflection`
- 지도별 방법

지도	방법	전환
Google 지도	API 승인 전: `business.google.com` 관리 화면 브라우저 자동화. 승인 후: Business Profile API(Business Information API `locations.patch`에 `updateMask`로 승인한 칸만 지정). 칸별 API 지원 범위는 D-14	환경변수 `GOOGLE_FIX_MODE=browser·api`
카카오맵	파트너센터 매장 > 매장정보 화면 브라우저 자동화(수정 API 없음)	—

- 자동화 차단(로그인 화면, 보안 확인, 캡차, 예상하지 못한 화면)이 나타나면 즉시 멈추고 다시 시도하지 않는다. 그 칸과 같은 정비안의 남은 칸을 `manual_required`로 바꾸고 관리자에게 수동 처리 이메일을 보낸다. 차단을 우회하는 설정·도구는 쓰지 않는다.
- 시간 초과 같은 일시 오류는 1번 다시 시도하고, 그래도 실패하면 `manual_required`로 넘긴다.
- 반영 확인: `pending_reflection` 칸은 주기 작업 `recheckPendingFixes`가 1시간마다 `verifyFix`를 등록해 24시간 동안 다시 읽는다(카카오 반영 30분~1시간, Google 검토). 반영 확인은 읽기만 하고 다시 쓰지 않는다. 24시간이 지나도 반영되지 않은 칸의 처리는 D-15.

- 수동 처리: A01은 `fixLogs.status = manual_required`인 칸을 바꿀 값·관리 화면 바로가기와 함께 보여 준다. 관리자가 직접 고친 뒤 [처리 완료]를 누르면 `manual_done`, `handledBy·handledAt`을 기록한다.
- 정비 완료: 칸 상태가 바뀔 때마다 `finalizeFix(fixRequestId)`가 승인된 정비안의 모든 칸이 끝났는지(`applied· manual_done· disconnected`) 확인한다. 모두 끝나면 `fixRequests.status = remeasure_scheduled`, `fixCompletedAt` 기록, 정비 완료 이메일, `remeasureSchedules(dueAt = fixCompletedAt + 14일)` 생성.
- 덮어쓰기 방지: `applyFix`는 정비안당 한 번만 실행된다(`fixPlanId` 중복 방지 키, `fixLogs`의 `fixPlanItemId` 고유 키). 정비 완료 뒤 지도 값을 다시 쓰는 작업은 없다.

F-11 재측정과 전후 비교 보고서

- 화면 경로: S16 `/fix/report`. API: `GET /api/fix/report`
- 예약: `remeasureSchedules`에 저장해 작업 서버가 다시 시작돼도 사라지지 않게 한다. 주기 작업 `dispatchRemeasures`(10분마다)가 `dueAt`이 지난 `scheduled` 예약을 조건부 UPDATE로 `started`로 바꾼 뒤 재측정을 시작한다(중복 시작 방지).
- A01 [지금 재측정]: `dueAt = 지금`, `triggeredBy = admin`. 보고서에는 정비 완료부터 재측정 시작까지의 일수(`intervalDays`)를 "정비 후 N일 만에 측정"으로 표시한다.
- 재측정 순서: `collectStore` → `runMeasurement(kind = remeasure, questionSetId = 정비 전 측정과 같음)` → `finalizeRun`(F-07 지표) → `compareFixValues` → `generateComparisonReport`
- `compareFixValues`: 정비 기록의 바꾼 값을 재측정 때 다시 모은 지도 값과 비교해 `fixLogs.remeasureValue`, `remeasureMatch(match·differs·unknown)`를 저장한다. Google은 Places 재수집 값, 카카오는 연결이 살아 있으면 파트너센터에서 다시 읽은 값을 쓰고, 읽지 못하면 `unknown`.
- 실패율이 30%를 넘으면 `fixRequests.status = remeasure_failed`로 두고 보고서를 만들지 않는다. 다시 시도는 F-06과 같이 하루 3회까지다.
- `generateComparisonReport(fixRequestId)`
  - 전후 지표: AI별 등장률, 평균 등장 순서, 질문별 등장 횟수, 우리 가게 추천에 쓰인 출처, Google 지도·카카오맵 인용 여부(출처 URL 도메인으로 판정)
  - 주변 가게: 정비 전 상위 5곳의 전·후 등장률(재측정 쪽은 `recommendations`에서 정규화 이름으로 다시 계산)과 정비 후 새 상위 5곳
  - 한쪽 측정에서 "측정하지 못함"인 질문은 비교에서 빼고 표시한다.
  - 보고서 문장은 계산값과 고정 문구 템플릿으로 만든다. 순위·노출 보장 표현이 들어가지 않게 LLM 생성 문장은 넣지 않는다.
  - 생성 목표 3분. 실패하면 1번 자동 재시도, 그래도 실패하면 `comparisonReports.status = generation_failed`, A01 [다시 생성].
- 완료되면 `comparisonReports`(정비 전·후 측정 ID, 정비 기록 참조, 내용 JSONB)를 저장하고 `fixRequests.status = report_ready`, 보고서 완료 이메일을 보낸다.

F-12 이메일 알림

- 이벤트가 생기면 `sendEmail` 작업을 등록한다. 템플릿은 기획서 F-12 표의 10종이다.

템플릿	발송 시점	받는 사람
`verify_email`	이메일 가입, 카카오 이메일 입력	사장님
`password_reset`	재설정 요청(소셜 가입 계정은 가입 방식 안내)	사장님
`diagnosis_result`	무료 진단 `completed·failed`	사장님
`fix_requested`	자동 정비 신청 `requested`	관리자
`fix_decided`	신청 승인·거절	사장님
`map_connection`	지도 `connected`, 초대 미도착 3일	사장님
`fix_plan_ready`	정비안 `awaiting_approval`	사장님
`manual_required`	칸이 `manual_required`가 됨(같은 정비안은 한 번에 묶어 발송)	관리자
`fix_completed`	정비 완료(`remeasure_scheduled`)	사장님
`report_ready`	전후 비교 보고서 완료	사장님

- 실패 시 3번까지 다시 시도하고 `emailLogs`에 결과를 남긴다. 발송 실패는 측정·정비·보고서 상태에 영향을 주지 않는다.

- 사장님 메일에는 로그인이 필요한 화면 링크와 기획서 F-12 표의 요약만 넣는다. 정비 완료 메일의 바꾼 내용은 지도별 칸 이름으로 요약하고, 바꾼 값 원문과 고쳐야 할 점 내용은 넣지 않는다.

4. 외부 연동 명세

서비스	용도	인증	제한·저장 조건	실패 처리
Google Places API(New)	가게 식별, 상세정보, 재측정 때 반영 확인, (P1) 메뉴판 사진	API 키(서버)	장소 ID 외 장기 저장 제한 확인 필요	5초 제한, 1회 재시도. 식별 실패는 "찾지 못함", 상세정보 실패는 "수집 실패"
카카오 로컬 API	가게 식별, 기본정보	REST API 키	장소 ID·장소 링크만 저장 가능, 좌표 저장 불가	5초 제한, 실패 시 "찾지 못함"
NAVER API HUB 지역 검색	가게 식별, 홈페이지 주소 확인, 기본정보 비교	NCP API Gateway 키	월 77.5만 건·키당 50 RPS(현재 한시 무료), AI 활용 조건 확인 전 AI 입력 금지	5초 제한, 실패 시 "찾지 못함"
OpenAI API	측정, 추천 추출, 질문 생성, 정보 수집용 웹 검색, 정비안 문장 생성	API 키	모델 ID는 사전 확인(PoC)에서 확정	60초 제한, 2회 재시도
Gemini API	측정, 정보 수집용 웹 검색	API 키	Google 검색 그라운딩 요금은 건별 과금	60초 제한, 2회 재시도
인스타그램 Graph API	Business Discovery(PoC 통과 시)	seelo 비즈니스 계정 토큰	앱 심사·비즈니스 인증 필요 여부 PoC 확인	실패 시 건너뛰고 AI 웹 검색 결과만 사용
Google 비즈니스 프로필 관리 화면(`business.google.com`)	API 승인 전 초대 수락, 연결 확인, 현재 값 읽기, 정비, 반영 확인	seelo 공용 Google 계정(2단계 인증), 브라우저 자동화(D-13)	사장님이 초대한 가게·승인한 칸만. 자동 접근에 따른 계정 정지 위험(기획서 14장)	차단은 즉시 중단·수동 처리, 일시 오류는 1회 재시도 후 수동 처리
Google Business Profile API	API 승인 후 초대 수락, 상태 확인(`getVoiceOfMerchantState`), 현재 값 읽기, 정비, 반영 확인	OAuth 2.0(seelo 공용 Google 계정의 토큰)	사용 신청·승인 필요(기획서 15장). 승인 전에는 끔	1회 재시도 후 수동 처리. 403·404는 권한 회수로 처리
카카오비즈니스 파트너센터	초대 수락, 연결 확인, 현재 값 읽기, 정비, 반영 확인	seelo 공용 카카오계정(2단계 인증), 브라우저 자동화(D-13)	수정 API 없음. 외부 운영자·자동화 허용 여부는 사전 확인(PoC) 때 고객센터에 확인. 반영 30분~1시간	Google 관리 화면과 같음
이메일 서비스	알림 발송	SMTP 계정	발신 도메인 SPF·DKIM 설정	3회 재시도

- 모든 키·계정 접속 정보·토큰은 환경변수로 넣고 저장소에 올리지 않는다.

5. 비동기 작업

작업	큐	시작 조건	제한 시간	재시도	중복 방지 키
`collectStore`	collect	가게 식별 완료, 정비 신청 승인	30초 목표	출처별 1회	`storeId+runId`

작업	큐	시작 조건	제한 시간	재시도	중복 방지 키
		(진단 없는 가게), 재측정 시작			
`generateQuestions`	measure	수집 완료(진단만, 재측정은 건너뜀)	60초	3회 생성 후 템플릿	`runId`
`runMeasurement`	measure	질문 확정, 다시 시도, 재측정 수집 완료	20분	없음(하위 작업이 재시도)	`runId`
`callAi`	ai_openai·ai_gemini	`runMeasurement`	60초	2회	`runId+questionId+provider+re`
`extractRecommendations`	measure	`callAi` 성공	60초	1회	`responseId`
`finalizeRun`	measure	모든 `callAi` 종료	60초	1회	`runId`
`verifyConnection`	fix	관리자 [연결 확인], API 초대 수락(승인 후)	3분	일시 오류 1회	`connectionId`
`readCurrentValues`	fix	연결됨	5분	일시 오류 1회	`connectionId`
`generateFixPlan`	plan	현재 값 읽기 완료(진단 없는 가게는 측정 완료 후), 관리자 [다시 생성]	3분 목표	1회	`fixRequestId+provider+versio`
`applyFix`	fix	정비안 승인	15분	일시 오류 1회, 자동화 차단은 없음	`fixPlanId`
`verifyFix`	fix	`recheckPendingFixes`	3분	없음(다음 시각에 다시 확인)	`fixLogId+회차`
`finalizeFix`	fix	칸 상태 변경(자동·수동·연결 끊김)	60초	1회	`fixRequestId`
`compareFixValues`	report	재측정 `finalizeRun` 완료	5분	1회	`fixRequestId+runId`
`generateComparisonReport`	report	`compareFixValues` 완료, 관리자 [다시 생성]	3분 목표	1회	`fixRequestId+version`
`sendEmail`	email	알림 이벤트	30초	3회	`emailLogs.id`
`cleanupUnverified`	주기 작업	매시간	—	—	—
`purgeSnapshots`	주기 작업	매일	—	—	—
`checkPendingInvites`	주기 작업	매시간	—	—	—
`recheckPendingFixes`	주기 작업	매시간(칸별 24시간까지)	—	—	—
`dispatchRemeasures`	주기 작업	10분마다	—	—	`remeasureSchedules.id`
`retryStalledJobs`	주기 작업	10분마다	—	—	—

- 큐 도구는 Procrastinate다. 작업 정의와 대기열은 PostgreSQL에 저장되므로 서비스 DB와 같이 백업된다.

- 중복 방지 키는 Procrastinate `queueing_lock`으로 넣어 같은 키의 대기 작업이 하나만 있게 한다. `queueing_lock`은 대기 중인 작업만 막으므로, 한 번만 실행돼야 하는 작업(`callAi`, `applyFix`, 재측정 시작)은 6장의 고유 키와 조건부 UPDATE로도 막는다.
- 재시도는 Procrastinate 재시도 전략(`RetryStrategy`, 지수 백오프)으로 둔다.
- 제한 시간은 작업 코드 안에서 건다(외부 호출 제한 시간, `asyncio.timeout`).
- 주기 작업은 워커가 등록하고, 워커가 여러 개여도 주기마다 한 번만 등록된다. 워커가 하나도 없으면 등록되지 않으므로 운영 환경에서는 워커를 항상 하나 이상 띄운다.
- 취소(가게 변경 F-02, 탈퇴 F-01): 대기 중인 작업은 취소하고, 실행 중인 작업은 중단을 요청한다(`abort`). 실행 중인 작업은 스스로 멈춰야 하므로, 오래 걸리는 동기 작업(`runMeasurement`, `applyFix`)은 단계마다 `should_abort()`를 확인한다. 중단된 작업은 재시도하지 않는다.
- 워커가 강제 종료되면 작업이 실행 중 상태로 남는다. `retryStalledJobs`가 이런 작업을 다시 시도하고, 이미 끝난 호출은 6장 고유 키로 다시 부르지 않는다.

6. 데이터 모델

테이블	핵심 컬럼·관계
users	id, email, emailVerifiedAt, passwordHash, role, failedLoginCount, lockedUntil, createdAt
authIdentities	id, userId, provider(email· google· kakao), providerUserId, createdAt
authTokens	id, userId, type(verify·reset), tokenHash, expiresAt, usedAt
sessions	id, userId, tokenHash, csrfTokenHash, expiresAt, createdAt, lastSeenAt
rateLimits	key, windowStart, count — 가입·비밀번호 재설정 요청 제한(9장). Redis를 쓰지 않으므로 DB에 센다
freeUsageLedger	emailHash, usedAt
stores	id, ownerId, name, inputUrl, inputProvider, category, status, archivedAt, createdAt
storeSources	id, storeId, provider(google·kakao·naver·website·instagram), externalId, url, matchStatus, matchedAt
sourceSnapshots	id, sourceId, runId, collectedAt, payload(JSONB), expiresAt — 카카오 로컬 API 값은 만들지 않음
webEvidence	id, storeId, runId, provider(openai·gemini), siteUrl, siteName, fields(JSONB), collectedAt
questionSets	id, storeId, version, createdAt
questions	id, questionSetId, lang, intentType, text, orderNo
measurementRuns	id, storeId, questionSetId, kind(diagnosis·remeasure), fixRequestId, isFree, status, progressDone, progressTotal, failureRate, costUsd, startedAt, completedAt
aiExecutions	id, runId, questionId, provider, modelId, repeatNo, status, attempt, latencyMs, errorCode
aiResponses	id, executionId, rawText, rawJson(JSONB), sourceUrls
recommendations	id, responseId, name, normalizedName, rank, sourceUrls, isTarget
metrics	id, runId, provider, questionId(전체면 null), subjectType(target· competitor), subjectName, metricName, value, denominator

테이블	핵심 컬럼·관계
findings	id, runId, fixPlanId(plan 단계), stage(diagnosis·plan), category, titleKey, provider(google·kakao), field, competitorCount, questionCount, priority, evidence(JSONB)
fixRequests	id, userId, storeId, baseRunId, status, rejectReason, decidedBy, decidedAt, fixCompletedAt, createdAt
mapConnections	id, fixRequestId, provider(google·kakao), status(not_connected·invited attentionReason, invitedAt, inviteReminderSentAt, connectedAt, confirmedBy, revokedAt
fixPlans	id, fixRequestId, provider, baseRunId, version, status(generating·gener createdAt, approvedAt, approvedBy
fixPlanItems	id, fixPlanId, field, currentValue, proposedValue, approvedValue, reason, findingIds, evidence(JSONB), needsOwnerInput, sensitive, ownerConfirmed, ownerEdited, excluded
fixLogs	id, fixPlanItemId(고유), provider, field, method(browser· api· manual), beforeValue, appliedValue, status(queued·applied·pending_reflection·manual_required·mar attempt, errorCode, manualReason, appliedAt, verifiedAt, verifyCount, handledBy, handledAt, remeasureValue, remeasureMatch
remeasureSchedules	id, fixRequestId, dueAt, triggeredBy(scheduler·admin), status(scheduled·started·canceled), runId, createdAt
comparisonReports	id, fixRequestId, storeId, baseRunId, afterRunId, intervalDays, version, status(generating· completed· generation_failed), content(JSONB), createdAt
emailLogs	id, userId, type, status, attempt, sentAt, error
auditLogs	id, actorId, action, targetType, targetId, createdAt

- 표의 컬럼 이름은 API 필드 이름(`camelCase`)이다. DB 컬럼은 `snake_case`로 만든다(1.1).
- 테이블별 전체 DDL·제약·ERD와 이 표에 없는 보조 테이블·컬럼은 백엔드 구현계획 02에 있다. 백엔드 구현 순서와 로직은 백엔드 구현계획을 따른다.
- 사용자·가게·측정·정비 관계는 정규화된 테이블로 관리하고, 외부 원본 응답, 정비 근거, 보고서 본문만 JSONB로 저장한다.
- 작업 큐 테이블(`procrastinate_` 접두사)은 Procrastinate가 제공하는 스키마·마이그레이션으로 만들고, 서비스 테이블의 Alembic 마이그레이션과 따로 관리한다.
- 탈퇴(F-01) 때 지울 행은 `users`를 참조하는 외래 키의 `ON DELETE CASCADE`로 한 트랜잭션에서 지운다. `freeUsageLedger`는 `users`를 참조하지 않는다.
- 정비 신청은 가게당 진행 중이거나 끝난 것 1건만 둔다(`fixRequests`에 `status <> 'rejected'` 조건의 `storeId` 부분 고유 인덱스).
- 정비안의 칸별 값: `currentValue`(연결 뒤 읽은 현재 값), `proposedValue`(seelo가 만든 바꿀 값), `approvedValue`(사장님이 승인한 값). 정비 기록(`fixLogs`)은 고치기 전 값(`beforeValue`), 바꾼 값(`appliedValue`), 반영 확인(`verifiedAt`·`remeasureMatch`)을 남긴다.
- 수동 처리 칸은 `fixLogs.status = manual_required`로 관리하고, A01 수동 처리 목록은 이 조건으로 조회한다.
- 유료 보고서용으로 설계했던 `reportRequests`·`entitlements`·`reports`와 이전 `improvementVersions`는 만들지 않는다(기획서 15장, 2026-10-17).

7. 상태 모델

가게

```
collecting → measuring → diagnosed
     │            └→ measure_failed → (다시 시도) → measuring
     └→ collected(무료 진단을 이미 쓴 계정) → (정비 신청 승인) → measuring
변경 시 → archived
```

- 후보 선택(S09)은 가게를 저장하기 전 단계라 가게 상태에 넣지 않는다.
- 수집에서 지역·업종을 얻지 못하면 `measure_failed`가 된다.

측정 작업

```
queued → running → parsing → completed
                          └→ failed(실패율 30% 초과)
어느 단계에서든 가게 변경·중단 → canceled
```

정비 신청

```
requested → approved → connecting → fixing → remeasure_scheduled → remeasuring → report_ready
    └→ rejected → (사장님 다시 신청: 새 requested 행)
remeasuring → remeasure_failed → (다시 시도) → remeasuring
```

- `approved` → `connecting`: 기준 진단(`baseRunId`)이 있으면 바로, 없으면 측정이 `completed`된 뒤
- `connecting`(연결·정비안 생성·승인 대기) → `fixing`: 첫 정비안이 승인될 때
- `fixing`(승인한 칸 정비, 수동 처리 포함) → `remeasure_scheduled`: 정비 완료(F-10), 14일 대기
- `remeasure_scheduled` → `remeasuring`: 예약 시각 도래 또는 A01 [지금 재측정]. 보고서 생성까지 `remeasuring`에 둔다.
- 지도별 진행은 `mapConnections·fixPlans·fixLogs` 상태로 따로 관리한다.

지도 연결·정비안·칸

```
[연결]
not_connected → invited → connected
                   └→ needs_attention(인증 필요 등) → (조치 후 연결 확인) → connected
connected → revoked(권한 회수) → (다시 초대) → invited

[정비안]
generating → awaiting_approval → approved
    └→ generation_failed → (다시 생성) → generating

[칸]
queued → applied
    ├→ pending_reflection → applied(반영 확인) / D-15
    └→ manual_required → manual_done
어느 단계에서든 권한 회수 → disconnected
```

S07 홈 상태 카드와의 대응

현재 가게에 정비 신청이 있으면 신청 상태로 카드를 고르고, 없으면 가게 상태로 고른다. 위에서부터 처음 맞는 카드 하나를 보여 준다.

S07 카드	판단 기준
가게 없음	보관 중이 아닌 가게 없음
정비 신청됨	신청 `requested`
신청 거절됨	가장 최근 신청 `rejected`
수집·측정 중	신청 없음 또는 `approved`이면서 가게 `collecting·measuring`

S07 카드	판단 기준
측정 실패	신청 없음 또는 `approved`이면서 가게 `measure_failed`
진단 완료	신청 없음, 가게 `diagnosed`
무료 사용 후 가게 변경	신청 없음, 가게 `collected`
정비안 확인 필요	신청 `connecting·fixing`이고 `awaiting_approval` 정비안 있음
정비안 준비 중	신청 `connecting`이고 `generating·generation_failed` 정비안 있음
연결 필요	신청 `connecting`이고 정비안 없음
정비 중	신청 `fixing`
재측정 대기	신청 `remeasure_scheduled`
재측정 중	신청 `remeasuring`
재측정 실패	신청 `remeasure_failed`
보고서 완료	신청 `report_ready`

- 연결 끊김: 어느 카드든 `mapConnections.status = revoked`인 지도가 있으면 "Google(또는 카카오) 연결이 끊겼어요"와 [다시 연결하기]를 함께 보여 준다.
- 정비 중 카드의 "일부 칸은 seelo 팀이 직접 고치고 있어요"는 `manual_required` 칸이 있을 때 보여 준다.

A01 상태 필터와의 대응

A01 상태	판단 기준
대기	신청 `requested`
승인	신청 `approved`
연결 대기	신청 `connecting`이고 정비안 없음
정비안 확인 대기	`awaiting_approval` 정비안 있음
정비 중	신청 `fixing`
수동 처리 필요	`manual_required` 칸 있음
재측정 대기	신청 `remeasure_scheduled·remeasuring`
보고서 완료	신청 `report_ready`
거절	신청 `rejected`
실패	정비안· 보고서 `generation_failed` 또는 신청 `remeasure_failed`

8. 오류 처리

오류	사용자 처리	서버 처리
지원하지 않는 링크	S08 오류 문구	제공자 판별 실패 코드 기록, 가게 저장 안 함
가게 식별 불가·애매	후보 선택(S09) 또는 다른 링크 요청	자동 확정 금지
대상 업종 아님	S08 오류 문구	가게 저장 안 함
일부 출처 수집 실패	받은 값만 표시	출처별 성공·실패 저장
AI 호출 실패	진행 계속	미노출과 분리, 2회 재시도, 실패율 30% 초과 시 측정 실패(무료 차감 없음)

오류	사용자 처리	서버 처리
추천 추출 실패	표시 없음	원문 유지, 지표에서 제외하고 건수 기록
초대 미도착(3일)	S15 단계 안내 다시 표시, 이메일	`inviteReminderSentAt` 기록, 1번만 발송
연결 확인 불가(인증 필요·대표 권한 없음 등)	S15 카드 "권한 확인 필요"와 안내	`mapConnections.status = needs_attention`, 사유 저장
지도 권한 회수	S07·S15 "연결이 끊겼어요", 다시 연결하기	`revoked`, 그 지도의 남은 칸 `disconnected`, 정비 중단
정비안 생성 실패	S07 "정비안 준비 중" 유지	1회 재시도 후 `generation_failed`, 관리자 다시 생성
자동화 차단(로그인 차단·보안 확인)	S13 칸 상태 "seelo 팀 처리 중"	즉시 중단, 우회· 재시도 없음, `manual_required`, 관리자 이메일
정비 실패(저장 오류·화면 변경·시간 초과)	S13 칸 상태 "seelo 팀 처리 중"	1회 재시도 후 `manual_required`
반영 지연	S13 칸 상태 "반영 대기"	24시간 동안 1시간마다 다시 읽기(이후 처리는 D-15)
재측정 실패	S07 재측정 실패, 다시 시도 하루 3회	보고서 생성 안 함
보고서 생성 실패	S07 "재측정 중" 유지	1회 재시도 후 `generation_failed`, 관리자 다시 생성
권한 없음	로그인 안내 또는 S07	정비안·정비 기록·보고서 내용을 응답에 넣지 않음
외부 API 한도 초과(429)	진행 지연	대기 후 재시도, 관리자 오류 추적에 기록

9. 비기능 요구사항

보안

- Argon2id 비밀번호 해시, 토큰은 해시로만 저장
- CSRF 헤더 검사(2.1), 보안 쿠키, 서버 세션 14일
- 로그인 5회 실패 시 10분 잠금, 가입·비밀번호 재설정 요청은 IP당 분당 5회 제한
- API 키, seelo 공용 Google·카카오 계정 접속 정보, Google API 토큰은 환경변수로 관리
- 모든 객체에 소유자·관리자 접근 검사
- 로그에 비밀번호, 토큰, 이메일 원문, 계정 접속 정보를 남기지 않음
- 자동 정비는 사장님이 초대한 가게의 승인한 칸만 고치고, 자동화 차단을 우회하는 설정·도구를 쓰지 않음

안정성

- 외부 호출마다 제한 시간을 둔다(지도·웹사이트 5~10초, AI 60초, 관리 화면 작업 3~15분).
- 비동기 작업은 중복 방지 키를 쓴다.
- 일부 출처·공급자가 실패해도 성공한 결과는 보존한다.
- 실패율이 30%를 넘으면 진단·재측정 결과를 확정하지 않는다.
- 자동 정비는 칸마다 고치기 전 값을 먼저 저장한 뒤 수정한다.
- 재측정 예약은 DB에 두어 작업 서버가 다시 시작돼도 유지한다.

성능 목표

- 일반 화면의 API는 외부 API 호출 없이 2초 이내 응답
- 가게 식별 10초 이내, 정보 수집 30초 이내, 측정 5~15분, 정비안 생성 3분 이내, 전후 비교 보고서 생성 3분 이내
- 긴 작업은 바로 작업 ID를 돌려주고 진행률을 조회하게 한다.

추적성

- 질문 세트 버전, 모델 ID, 회차, 실행 시각, 답변 원문을 저장한다.

- 정비안의 칸마다 근거(고쳐야 할 점, 응답 ID, 수집값)로 이동할 수 있어야 한다.
- 정비 기록에 칸별 정비 방법(브라우저·API·수동), 고치기 전 값, 바꾼 값, 시각, 반영 확인 결과를 남긴다.
- 관리자 신청 승인·거절, 연결 확인, 수동 처리 완료, 다시 생성, 지금 재측정, 사장님 정비안 승인, 탈퇴, 가게 변경을 `auditLogs`에 남긴다.

비용

- 진단 1회 호출 수: 측정 60회 + 정보 수집용 AI 웹 검색 2회 + 추천 추출 최대 60회(저가 모델) + 질문 생성 1회 이상
- 측정 원가 추정은 약 1,400~2,500원이다(웹 검색 포함 저가 모델, 2026년 10월 공개 가격·환율 1,400원 기준). 추출·수집 호출은 별도다.
- 정비 1건: 정비 전·후 측정 2번(진단이 없던 가게도 승인 후 측정 1번 + 재측정 1번) + 지도별 정비안 생성
- `measurementRuns.costUsd`에 실제 비용을 기록하고, 하루 비용 상한(환경변수)을 넘으면 새 측정을 대기시킨다.

10. 핵심 인수 테스트

정상 흐름

- ☐ 이메일로 가입하고 인증 링크를 누르면 로그인할 수 있다.
- ☐ 구글·카카오로 가입·로그인할 수 있다. 카카오가 이메일을 주지 않으면 S04에서 인증 후 가입된다.
- ☐ 가게 이름과 Google·카카오·네이버 링크 중 하나로 가게가 식별되고, 나머지 지도에서도 같은 가게를 찾는다.
- ☐ 무료 진단을 쓰지 않은 계정은 수집이 끝나면 측정이 자동으로 시작되고, 성공한 호출의 원문이 모두 저장된다.
- ☐ 무료 진단에서 AI별 등장률·평균 순서·출처, 주변 가게 5곳 비교, 고쳐야 할 점 개수를 볼 수 있다.
- ☐ 자동 정비를 신청하면 A01에 나타나고, 관리자가 승인하면 사장님에게 연결 안내 이메일이 간다.
- ☐ 사장님이 Google·카카오에 seelo 계정을 초대하고 관리자가 [연결 확인]을 누르면 연결됨으로 바뀌고 정비안이 만들어진다.
- ☐ 사장님이 정비안을 확인하고 [정비 시작]을 한 번 누르면 승인한 칸만 Google·카카오에 반영되고, 칸별 고치기 전 값·바꾼 값·반영 확인이 기록된다.
- ☐ 정비가 끝나면 14일 뒤 재측정이 예약되고, A01 [지금 재측정]으로 앞당기면 같은 질문으로 재측정해 전후 비교 보고서가 만들어진다.
- ☐ 기획서 F-12의 이메일 10종이 해당 시점에 발송된다.
- ☐ 비밀번호 재설정 링크로 비밀번호를 바꿀 수 있다.

실패·경계 흐름

- ☐ 인증을 마친 이메일로 다른 방식 가입을 시도하면 기존 가입 방식이 안내되고 가입되지 않는다.
- ☐ 인증·재설정 링크는 만료 후나 두 번째 사용 시 동작하지 않는다.
- ☐ 로그인 5회 실패 시 10분 동안 잠긴다.
- ☐ 지원하지 않는 링크, 음식점·카페·바가 아닌 가게는 저장되지 않는다.
- ☐ 이름이 같은 가게가 여러 곳이면 자동 확정하지 않고 S09 후보를 보여 준다.
- ☐ AI 호출 실패는 미노출로 계산되지 않고, 실패율 30% 초과 시 무료 횟수가 차감되지 않는다.
- ☐ 등장률 0%는 오류가 아닌 정상 결과로 표시된다.
- ☐ 무료 진단을 쓴 뒤 가게를 바꾸거나 탈퇴 후 같은 이메일로 다시 가입해도 무료 진단을 다시 쓸 수 없다.
- ☐ 다른 사용자의 가게·측정·정비안·정비 기록·보고서 ID로 접근하면 거부된다.
- ☐ 정비 신청이 진행 중이면 가게를 바꿀 수 없다.
- ☐ 같은 가게로 정비가 진행 중이거나 끝났으면 다시 신청할 수 없고, 거절된 경우에만 다시 신청할 수 있다.
- ☐ 측정·재측정 실패 후 다시 시도는 가게당 하루 3회까지만 된다.
- ☐ 무료 진단(S11) 응답과 권한 없는 사용자 응답에 고쳐야 할 점·정비안·보고서 내용이 들어 있지 않다.
- ☐ 카카오 로컬 API 데이터는 장소 ID·링크 외에는 DB에 저장되지 않는다. 연결 뒤 파트너센터에서 읽은 값은 정비안·정비 기록의 칸 값으로만 저장된다.
- ☐ 정비안에 입력 사실에 없는 영업시간·가격·전화가 들어가면 `[ ]`와 "확인 필요"로 바뀌고, 그 칸을 채우거나 빼기 전에는 [정비 시작]이 막힌다.
- ☐ 가게 이름·주소·업종은 정비안에 들어가지 않는다.
- ☐ [정비 시작]을 두 번 눌러도 정비는 한 번만 시작된다.
- ☐ 자동화 차단·수정 실패가 나면 그 칸은 우회 없이 "수동 처리 필요"로 넘어가고 관리자에게 이메일이 간다.
- ☐ 지도에서 seelo 권한을 회수하면 그 지도의 남은 칸은 고치지 않고 S07에 "연결이 끊겼어요"가 보인다.
- ☐ 정비 완료 뒤에는 같은 칸을 다시 쓰지 않는다(사장님이 바꾼 값을 덮어쓰지 않음).
- ☐ 재측정은 정비 전과 같은 질문 세트로 실행되고, 실패율 30% 초과 시 보고서를 만들지 않는다.
- ☐ 전후 비교 보고서에 순위·노출 보장 표현이 없다.

11. 현재 구현 상태

확인 완료(`04_MVP/hanoon-mvp`)

- 더미데이터 기반 랜딩·입력·진단·개선안·영문 페이지·전후 비교 화면(Flask·Jinja, 베타 스택과 다름). 영문 페이지는 기획서 P2(공개 영문 소개 페이지)이고, 전후 비교 화면은 더미 데이터 예시로 실제 재측정(F-11)과 연결되어 있지 않다.
- 등장률·평균 순서·정보 정확도·조건 반영률 계산
- 누락 정보·채널 불일치 탐지
- 기존 자동 테스트 14개 통과

미구현

- React 화면, FastAPI API(기존 MVP 계산 로직 `core/visibility.py`는 옮겨 쓸 수 있음)
- 실제 가입·로그인·이메일 인증·소셜 로그인
- 지도 링크 식별과 실제 정보 수집, 공식 웹사이트·AI 웹 검색 수집
- 제품 내부 OpenAI·Gemini API 연동
- PostgreSQL·마이그레이션·비동기 작업 큐
- 자동 정비 신청·관리자 승인, Google·카카오 연결
- 정비안 생성·승인, 자동 정비(브라우저 자동화·Business Profile API), 수동 처리
- 재측정 예약·전후 비교 보고서
- 이메일 알림

기존 테스트 통과는 더미 응답 기반 계산 로직 검증이며 실제 AI 노출 개선 효과를 의미하지 않는다.

12. 기술 결정 사항과 미정 사항

결정 사항

항목	결정
이전 D-01~D-06(지도, 로그인, 질문·반복 수, 주변 가게, 무료 진단, 베타 결제)	기획서 15장 결정 사항으로 확정
상품 구조·정비 대상·정비 방법	기획서 15장 결정 사항과 "지도별 정비 방법"을 따른다(2026-10-17)
화면	React + TypeScript + Vite SPA(2026-10-23)
API	FastAPI + Pydantic v2. 화면과 같은 도메인에서 `/api`로 서비스(2026-10-23)
데이터베이스	PostgreSQL + SQLAlchemy 2.0 + Alembic. DB 컬럼은 `snake_case`, API 필드는 `camelCase`(2026-10-23)
비동기 작업 도구	Procrastinate(PostgreSQL 기반 작업 큐·주기 작업). Redis와 별도 scheduler 프로세스는 쓰지 않음(2026-10-23, 이전 결정 Redis + RQ를 대체)
세션·CSRF	서버 세션(`sessions` 테이블) + CSRF 헤더 검사(2.1, 2026-10-23)
비밀번호 해시	Argon2id
OAuth 라이브러리	Authlib
파일 저장소	P0에서는 쓰지 않음

미정 사항

ID	항목	권장 초안	정할 때
D-07	배포 환경	서울 리전 VM 1대에 Docker Compose(`web`·`api`·`worker`·PostgreSQL). `fix` 워커는 관리자가 로그인한 브라우저 프로필	W-11-2 전

ID	항목	권장 초안	정할 때
		(D-13)을 디스크에 계속 둬야 하므로, 재시작 때 디스크가 초기화되는 관리형 컨테이너 서비스는 맞지 않는다. 관리형 PostgreSQL은 선택	
D-08	이메일 발송 서비스	SPF·DKIM을 지원하는 트랜잭션 이메일 서비스(Resend 또는 Amazon SES). 개발 환경은 Mailpit	W-10-1 전
D-09	AI 모델 ID	측정은 웹 검색을 지원하는 저가 모델(OpenAI mini 계열, Gemini Flash-Lite 계열), 정비안 문장 생성은 품질을 보고 결정	측정은 W-04-2 전, 정비안은 W-09-2 전
D-10	수집 데이터 보관 기한	기획서 15장 미정 3과 함께 결정. 연결 뒤 관리 화면에서 읽은 값의 저장 범위 포함	W-03-1 전
D-11	이름 유사도 기준값, 업종 목록	사전 확인(PoC) 정확도 측정 후 결정	W-02-3·W-02-4 전
D-12	하루 API 비용 상한 금액	팀 예산에 맞춰 결정(환경 변수로 설정)	W-04-2 전
D-13	브라우저 자동화 도구와 로그인 방식	Playwright(Python). 관리자가 2단계 인증으로 한 번 로그인한 브라우저 프로필을 `fix` 워커가 다시 쓰고, 로그인이 풀리거나 보안 확인이 나오면 멈추고 수동 처리	사전 확인(PoC) 후, W-09-5·W-09-6 전
D-14	Business Profile API 칸별 수정 방법	API 승인 뒤 칸(소개·영업 시간·웹사이트·전화·메뉴·편의 정보)별 지원 범위를 확인. 지원하지 않는 칸은 관리 화면 자동화 유지	API 승인 후
D-15	24시간 뒤에도 반영되지 않은 칸 처리	"수동 처리 필요"로 넘겨 관리자가 확인	W-09-7 전
D-16	한쪽 지도만 승인한 뒤 정비 완료·재측정 예약 시점	승인한 정비안의 칸이 모두 끝나면 정비 완료로 보고 재측정을 예약. 재측정 전에 다른 지도의 정비가 끝나면 그 완료일 + 14일로 다시 미룸	W-09-7 전
D-17	지도별 칸 사전(칸 이름·글자 수·형식)	사전 확인(PoC)에서 실제 관리 화면으로 확인. 카카오 한 줄 소개 50자·태그 5개는 기획서 기준	W-09-2 전

- 사전 확인(PoC) 항목은 기획서 13장을 따른다. 결과가 나오면 4장 외부 연동 명세와 이 표를 함께 고친다.

13. 작업별 기술 완료 조건

기획서 11장의 작업(W-ID)마다 기능 완료 조건에 더해 아래 기술 조건을 만족해야 완료로 본다. 설계 작업(W-01-1, W-02-1, W-04-5, W-06-1, W-08-3, W-09-3, W-12-3)은 기술 조건이 없다.

13.1 공통

- 작업마다 자동 테스트를 추가한다. API·워커는 `pytest`(httpx 외부 호출은 respx로 대체), 화면은 Vitest + Testing Library로 테스트한다. 외부 API는 녹화한 응답(fixture)으로 대체해 CI에서 돌린다.
- DB가 관련된 테스트는 실제 PostgreSQL(테스트용 컨테이너)에서 돌린다. 부분 고유 인덱스, 조건부 UPDATE, 동시 요청은 다른 DB로 재현되지 않는다.
- 작업 등록 여부는 Procrastinate 인메모리 커넥터(`InMemoryConnector`)로 확인하고, 중복 실행 방지·주기 작업 중복 등록 테스트는 실제 PostgreSQL에서 한다.
- 관리 화면 자동화는 저장해 둔 관리 화면 HTML(fixture)을 로컬에서 띄워 테스트한다. 로그인 화면·보안 확인 화면 fixture도 함께 둔다.
- 실제 외부 API를 부르는 시험은 `-m live` 표시로 분리하고, 테스트 가게 세트(기획서 11.1)로만 실행한다. 실제 지도를 고치는 시험은 동의하고 seelo를 초대한 지인 가게에서만 실행한다.
- 시험 결과(정확도, 시간, 비용)는 `03_실험/` 아래 작업 ID 이름의 기록 파일로 남긴다.

13.2 작업별 조건

ID	기술 확인 방법
W-01-2	토큰 원문·비밀번호 원문이 DB에 없음을 테스트로 확인. 인증 토큰 만료·재사용 거부 테스트. 미인증 계정 정리 작업 테스트
W-01-3	OAuth 콜백 `state` 검증. 제공자 응답을 fixture로 바꿔 중복 차단 3가지 조합 테스트
W-01-4	잠금 횟수· 해제 시각 테스트. 세션 쿠키 `Secure`·`HttpOnly`·`SameSite` 속성 확인. `sessions`에 토큰 원문이 없음. 로그아웃 후 세션 행 삭제와 보호 API 401 응답. CSRF 헤더가 없거나 다른 상태 변경 요청 403
W-01-5	가입 여부와 관계없이 같은 응답 본문·상태 코드. 요청 제한 초과 시 429
W-01-6	탈퇴 후 해당 사용자 ID로 남은 행이 없음을 테이블 전체 조회로 확인(`freeUsageLedger` 제외). 진행 중 Procrastinate 작업(대기 작업 취소, 실행 중 작업 중단 요청)과 재측정 예약 취소 확인
W-02-2	링크 형식별 단위 테스트(테스트 가게 20곳 × 3사 × 긴·짧은 링크 + 거부 10개). 짧은 링크 처리 시 리다이렉트 자동 추적을 하지 않음
W-02-3	`-m live` 정확도 시험 결과 기록. 외부 API 시간 초과를 흉내 낸 테스트에서 10초 안에 오류 응답
W-02-4	업종 목록(D-11) 단위 테스트. 대상이 아닌 업종 5곳 fixture 거부
W-02-5	보관 중이 아닌 가게 1곳 부분 고유 인덱스 위반 테스트. 정비 신청 진행 중(7장) 변경 요청 시 409
W-03-1	Places 응답 fixture로 필드 매핑 테스트. 출처 하나 실패 시 나머지 저장 테스트
W-03-2	카카오 수집 후 `sourceSnapshots`에 카카오 행이 없음. AI 요청 본문 로그에 네이버 응답 필드가 없음(테스트에서 요청 본문 검사)
W-03-3	robots.txt 금지 사이트 fixture에서 본문 요청이 0회. 크기·시간 제한 테스트

ID	기술 확인 방법
W-03-4	구조화 출력 JSON 스키마 검증. 한쪽 공급자 실패 fixture 테스트
W-03-5	기능 플래그 꺼짐 상태에서 Graph API 호출 0회. PoC 결과 기록
W-04-1	가게 이름 포함 질문을 거부하는 검증 함수 단위 테스트. 3회 실패 후 템플릿 사용 테스트
W-04-2	(`runId`, `questionId`, `provider`, `repeatNo`) 고유 제약 위반 테스트. 재시도 2회 후 실패 기록. `costUsd` 누적 확인
W-04-3	진행률 API 응답 시간 200ms 이내. 워커 강제 종료 후 `retryStalledJobs`로 다시 실행할 때 완료된 호출을 다시 부르지 않음
W-04-4	실패율 경계값(29%·30%·31%) 단위 테스트. 가게당 하루 다시 시도 횟수 4번째 요청 시 429
W-05-1	추출 결과 JSON 스키마 검증. 추출 실패 응답이 분모에서 빠지는지 테스트
W-05-2	이름 정규화 함수 단위 테스트(대소문자, 공백, 기호, 지점명, 한글·영문)
W-05-3	손 계산 예시 3개 fixture와 `metrics` 결과 비교 테스트
W-05-4	동률·5곳 미만·지점명 묶기 단위 테스트
W-06-2	무료 진단 API 응답 JSON에 `findings` 내용 필드가 없음을 테스트로 확인(S11 응답 모델에 해당 필드 없음)
W-06-3	7장 S07 카드 15가지와 연결 끊김 fixture로 카드 판정(API, `pytest`)과 카드·버튼 렌더링(Vitest) 테스트
W-07-1	분류 5가지 각각 판정·미판정(정보 없음) 단위 테스트. 네이버·카카오는 이름·주소·전화·업종만 "출처끼리 정보 다름"에 반영되고, AI 요청 본문에 네이버 응답 필드가 없음. Google·카카오 밖 항목(인스타그램 등)은 만들지 않음
W-07-2	동률·근거 없음 경계값 단위 테스트
W-08-1	신청 상태 전이 테스트(허용되지 않는 전이는 409). 진행중·완료 신청이 있는 가게의 새 신청 409(부분 고유 인덱스), 거절 후 새 신청 허용
W-08-2	`/admin` 경로 사장님 계정 403. 처리된 신청 재처리 409. 승인·거절·연결 확인·처리 완료·다시 생성·지금 재측정의 `auditLogs` 기록 확인
W-09-1	연결 상태 전이 테스트. `invitedAt` + 3일 fixture에서 미도착 이메일 1번만 등록. 권한 회수 fixture(관리 화면에 가게 없음, API 403· 404)에서 `revoked`와 남은 칸 `disconnected`. 읽은 현재 값이 `fixPlanItems.currentValue`에 저장
W-09-2	틀린 값 20개 fixture를 모두 `[ ]`로 바꾸는 사실 검사 함수 테스트. 칸 사전에 없는 칸(이름·주소·업종) 출력은 버림. 글자 수 초과 시 다시 생성. 생성 실패 fixture로 재시도 1회 후 `generation_failed`. JSON 스키마 검증
W-09-4	"확인 필요" 빈 칸·민감 정보 미확인 승인 요청 400. 동시 승인 요청 2개 중 1개만 성공하고 `applyFix` 1번 등록. 사

ID	기술 확인 방법
	장님이 고친 값의 글자 수·형식 검사. 권한 없는 계정의 정비안 API 요청에 본문 내용 없음(401 또는 403)
W-09-5	관리 화면 fixture에서 칸 수정·다시 읽기 테스트. 로그인·보안 확인 화면 fixture에서 즉시 중단, 재시도 0회, `manual_required`. API 모드에서 `updateMask`에 승인 칸만 포함. 수정 요청보다 `beforeValue` 저장이 먼저임을 확인
W-09-6	W-09-5와 같은 차단·실패 테스트. `pending_reflection` 칸이 1시간마다 24시간 동안 다시 확인되고, 그동안 쓰기 요청 0회
W-09-7	모든 칸 자동 완료·일부 수동 처리·연결 끊김 3가지 fixture에서 `remeasure_scheduled`와 `remeasureSchedules.dueAt = fixCompletedAt + 14일` 1행. 정비 완료 뒤 같은 칸 쓰기 요청 0회
W-10-1	발신 도메인 SPF·DKIM 검사 통과. 발송 실패 시 3회 재시도 후 `emailLogs.status = failed`
W-10-2	이벤트 10종(F-12) 각각 `sendEmail` 작업 등록 테스트. 발송 실패가 측정·정비·보고서 상태에 영향 없음
W-11-1	10장 실패·경계 인수 테스트 전체 통과. 로그 샘플에서 이메일·토큰·계정 접속 정보 원문 검색 결과 0건
W-11-2	배포 환경에서 10장 정상 흐름 인수 테스트 통과(재측정은 [지금 재측정]). 외부 API 하나를 끈 상태에서 나머지 흐름 동작
W-12-1	`dueAt`이 지난 예약만 시작. 워커 2개 동시 실행에도 `dispatchRemeasures`가 주기마다 1번만 등록되고 재측정은 1번만 시작(조건부 UPDATE). 재측정 `questionSetId`가 정비 전 측정과 같음. 워커 재시작 후 예약 유지. 탈퇴 시 예약 `canceled`. [지금 재측정] 시 `triggeredBy = admin`, `intervalDays` 기록
W-12-2	손 계산 전후 예시 3개 fixture와 결과 비교. 한쪽 "측정하지 못함" 질문 비교 제외. 보고서 문구에 순위·노출 보장 표현이 없음을 금지어 검사로 확인. 생성 실패 fixture로 재시도 1회 후 `generation_failed`
W-12-4	권한 없는 계정의 보고서 API 요청에 본문 내용 없음(401 또는 403). 보고서가 없으면 API는 404, 화면은 S12로 이동